Advanced NLP — Assignment 2

Mixture-of-Experts Translation, Optimizers Written from Scratch, and Decoding Strategies

[Divyansh Atri] *[2024113001]*
October 2026

This report covers the three tasks of the assignment. **Part 1** compares five feed-forward (FFN) designs, one dense and four Mixture-of-Experts (MoE), inside a Transformer that translates Vietnamese and Japanese into English. **Part 2** implements AdamW, NAdamW, Lion, Muon and (bonus) Sophia by hand and compares them on one language model. **Part 3** implements greedy, beam, top-k, top-p and pure sampling decoding on the forward pass of a pretrained Pythia-160M and evaluates them on ROCStories. All jobs ran on the IIIT ADA cluster with one NVIDIA RTX 2080 Ti (11 GB) each. Every number below comes from the result files written by these runs. The trained checkpoints are public on Hugging Face:

Part 1 (checkpoint, tokenizer, code, full-test BLEU)		**Part 2** (checkpoint, config, metrics, generations, `model.py`)	
M1 dense	`Devatri/anlp-assignment2-part1-model1`	AdamW	`Devatri/anlp-assignment2-part2-adamw`
M2	`Devatri/anlp-assignment2-part1-model2`	NAdamW	`Devatri/anlp-assignment2-part2-nadamw`
M3	`Devatri/anlp-assignment2-part1-model3`	Lion	`Devatri/anlp-assignment2-part2-lion`
M4	`Devatri/anlp-assignment2-part1-model4`	Muon	`Devatri/anlp-assignment2-part2-muon`
M5	`Devatri/anlp-assignment2-part1-model5`	Sophia	`Devatri/anlp-assignment2-part2-sophia`

1 Part 1: Mixture-of-Experts FFN variants

1.1 Setup

Data. `belumind/en-vi-ja-curated-500k-triplets` (pinned revision) provides (vi, ja, en) triplets with official train/validation/test splits. After NFC normalisation, removal of training rows whose English target also occurs in validation or test, and removal of over-length examples (nothing is truncated; a triplet is dropped if either direction is too long), 446,202 / 24,789 / 24,791 triplets remain. Each triplet yields two examples, vi→en and ja→en, with the same English target. A 16,000-token byte-level BPE tokenizer (full byte alphabet, so no unknown tokens) is trained on 50,000 training triplets in all three languages.

Model. A decoder-only Transformer reads `[BOS] source [SEP] English [EOS]` with causal attention. The loss is the cross-entropy of the English tokens and EOS only; source positions are masked out. The model has $d = 512$, 8 heads, 6 pre-LayerNorm blocks, rotary position embeddings, GELU, dropout 0.1, bias-free linear layers, tied input/output embeddings and a 256-token context. The source language is never given to the model. The non-FFN backbone is initialised identically in all five variants.

Training. AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1 on matrices), length-bucketed batches of at most 8,192 padded tokens and 128 pairs, linear warm-up over the first 3% of tokens and cosine decay to 0.1×, gradient clipping at 1.0, FP16 autocast, seed 42. All five models train on *the same* 12,473 batches: 59,998,213 non-padding input tokens (≈1.56 epochs) and 29,239,313 supervised English tokens; the schedule digest is asserted identical. The peak learning rate was chosen per model from $\{3 \cdot 10^{-4}, 6 \cdot 10^{-4}\}$ with 1M-token pilot runs scored on validation NLL. All five chose $6 \cdot 10^{-4}$.

Evaluation. Test perplexity is teacher-forced over all 49,582 test pairs (1,038,360 target tokens). For BLEU, every model translates the *full* test split (24,791 triplets, 49,582 translations) with greedy decoding (KV cache, at most 160 new tokens), scored with sacreBLEU 2.6.0 (13a tokenisation, mixed case, exponential smoothing, one reference) and given a bootstrap 95% CI. During development a fixed 1,000-triplet sample was used; on it we also ran a *paired* bootstrap against M1 (300 resamples).

1.2 The five FFN variants and parameter matching

All MoE layers use a bias-free linear router, a softmax over all experts and then top-k selection. Top-2 weights are renormalised; top-1 uses the raw softmax probability so that the router still gets a gradient from the task loss. There is no capacity limit, so no token is dropped. Two auxiliary losses are averaged

Table 1: Part 1 results on the test split. Parameters in millions. PPL is target-token perplexity over all 49,582 test pairs. BLEU is on the full test split; "unseen" excludes the 1,097 test triplets whose vi or ja source occurs verbatim in training. "Min" is training wall-clock time.

M	FFN	Total	Active	PPL	PPL vi	PPL ja	BLEU	95% CI	vi→en	ja→en	unseen	Min
1	Dense	27.08	27.08	5.245	4.451	6.180	33.98	33.71–34.31	39.07	28.79	33.56	11.6
2	4E top-1	27.08	17.649–17.655	5.356	4.567	6.280	33.51	33.26–33.87	38.50	28.43	33.09	17.6
3	4E top-2	27.08	20.788–20.801	5.339	4.523	6.301	33.56	33.30–33.91	38.62	28.40	33.13	17.1
4	Shared+3E top-1	27.08	20.794–20.801	5.380	4.560	6.349	33.60	33.34–33.93	38.65	28.45	33.19	17.1
5	4E top-2 (act.)	39.65	27.08	5.165	4.387	6.080	34.13	33.85–34.47	39.03	29.14	33.69	17.5

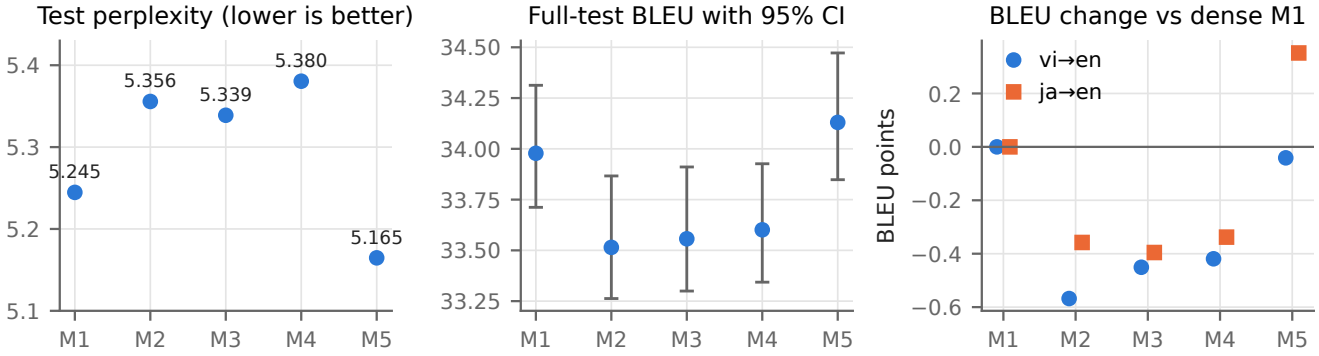


Figure 1: Part 1. Left: test perplexity. Middle: full-test BLEU with bootstrap 95% CI. Right: BLEU change against the dense model, per source language. The y-axes do not start at zero.

over layers: a Switch-style load-balancing loss $E \sum_e f_e \bar{P}_e$ (coefficient 0.01; f_e is the fraction of assignments to expert e, $\bar{P}_e$ its mean router probability) and a router z-loss (coefficient 0.001).

A dense FFN layer has $2 \cdot 512 \cdot 2048 = 2{,}097{,}152$ parameters. In **M2–M4** the router costs $4 \cdot 512 = 2{,}048$ parameters (in M4, a 3-way router plus a $512 \to 1$ sigmoid gate on the shared expert). The experts therefore share the remaining 2,046 hidden units, with widths $[512, 512, 511, 511]$. This gives exactly 2,097,152 parameters per layer and **27,079,680 in total, the same as M1**. In **M4**, expert 0 (width 512) is always active and scaled by its gate, and the router picks one of the other three. **M5** has four experts of width 1,023 with top-2: two active experts plus the router cost $2 \cdot 512 \cdot 2046 + 2048 = 2{,}097{,}152$ parameters per token, so its **active** count equals M1's 27,079,680 while its total is 39,650,304. Active counts in M2–M4 are small ranges because experts of width 511 and 512 are mixed (Table 1).

1.3 Results

Table 1 and Figure 1 give the results. Perplexity is computed over a million target tokens and gives a clear order: **M5 (5.165) < M1 (5.245) < M3 (5.339) < M2 (5.356) < M4 (5.380)**. The same order holds for both source languages.

Same total parameters (M2–M4 vs. M1). All three MoE models are slightly worse than the dense model: perplexity is 1.8–2.6% higher and full-test BLEU 0.38–0.46 lower. They also use 23–35% fewer parameters per token (17.65–20.80M active vs. 27.08M), so when total size is fixed, routing saves compute per token at a small cost in quality. M2, with 35% fewer active parameters, loses only 2.1% perplexity. Between the two top-1 models, adding the always-on shared expert (M4) did *not* help here: M4 has more active parameters than M2 but the highest perplexity. Top-2 routing (M3) beats top-1 (M2) in perplexity, in line with its larger active count. The BLEU differences among M2–M4 (33.51–33.60) are far smaller than their CIs.

Same active parameters (M5 vs. M1). M5 is the best model on every metric: perplexity falls by 1.5% (vi 4.451→4.387, ja 6.180→6.080) and BLEU rises from 33.98 to 34.13. Almost all of the BLEU gain is on the harder ja→en direction (+0.35, vs. −0.04 for vi→en). With the per-token compute held fixed, the 46% extra (inactive) capacity helps.

Significance. The BLEU gaps are small. On the 1,000-triplet sample, the paired bootstrap differences against M1 are M2 -0.30 $[-0.99, 0.45]$, M3 -0.68 $[-1.33, 0.15]$, M4 -0.28 $[-0.95, 0.52]$ and M5 $+0.24$ $[-0.34, 1.02]$; every interval contains zero. The full-test CIs of M1 and M5 also overlap those of M2–M4. We therefore read the perplexity ordering as the reliable signal and the BLEU ordering as consistent with it but not statistically separated. Removing training-overlap sources ("unseen") lowers

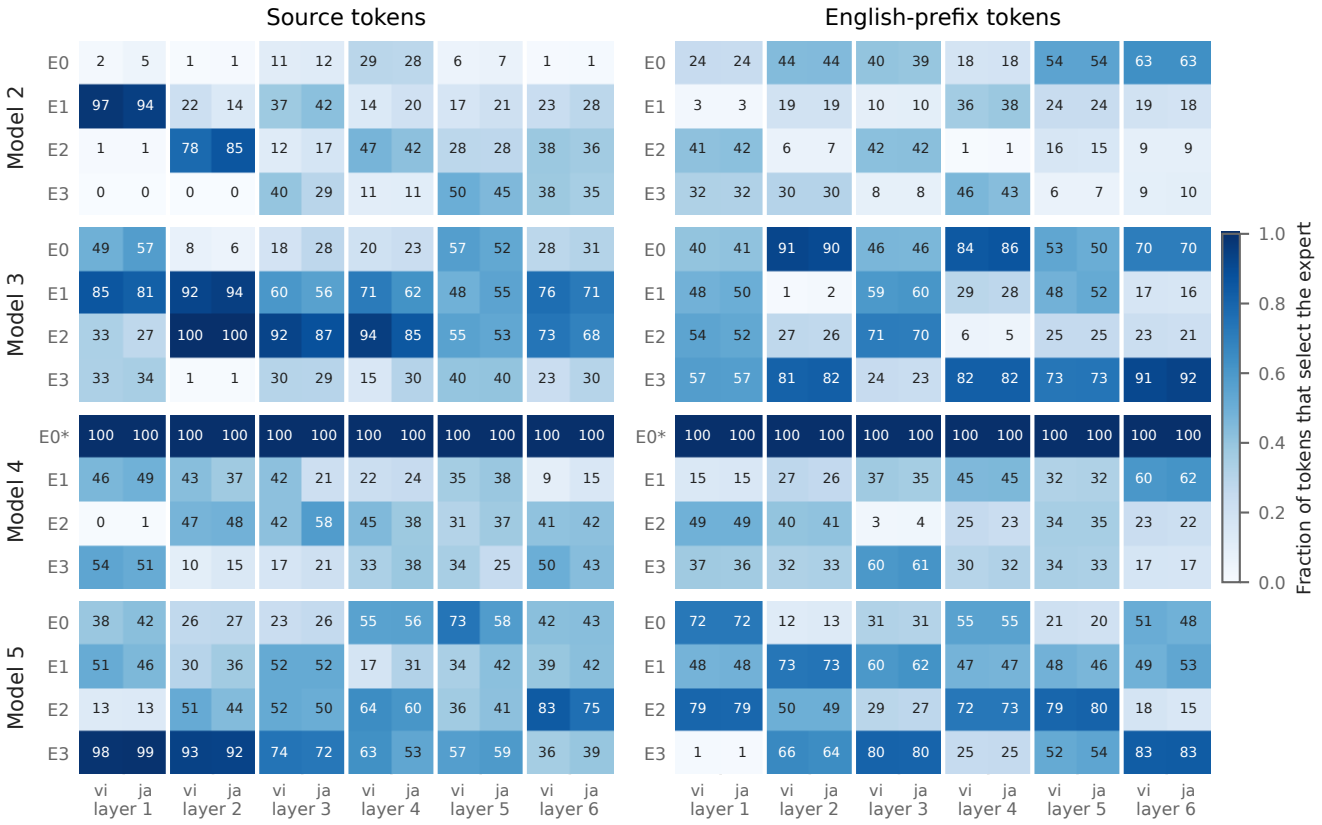


Figure 2: Expert selection rate (%) per layer, source language and token region, on 1,000 test pairs (about 21k source and 20k English-prefix tokens, teacher forced). For top-2 models a column sums to 200%. In M4, E0* is the shared expert (100% by construction).

Table 2: Routing diagnostics. H is the routing entropy normalised by ln(#routed experts) (1 = uniform), shown for layer 1 and averaged over layers and languages. "Max share" is the largest fraction of all routed assignments taken by one expert in any layer (the maximum is 0.5 under top-2). "Unused" counts (layer, language, region) cells in which some expert is never selected.

	Source tokens			English-prefix tokens			
M	H layer 1	mean H	max share	H layer 1	mean H	max share	Unused
2	0.15	0.67	0.97	0.84	0.82	0.63	0
3	0.94	0.87	0.50	0.99	0.88	0.46	0
4	0.66	0.88	0.58	0.91	0.91	0.62	0
5	0.86	0.94	0.49	0.80	0.91	0.42	0

every model's BLEU by about 0.42–0.44 and leaves the ranking unchanged.

Other observations. (i) ja→en is much harder than vi→en for every model (BLEU ≈28.4–29.1 vs. ≈38.5–39.1; perplexity ≈6.1–6.3 vs. ≈4.4–4.6). This is consistent with Japanese differing from English in script and word order (SOV vs. SVO), while Vietnamese is SVO and Latin-script. (ii) Validation NLL fell at every one of the 12 evaluations for every model. Validation and test NLL agree within 0.004, so no model overfits in 1.56 epochs; all are still data-limited. (iii) Every model picked the upper pilot LR (grid edge), so a larger one might do better. (iv) The MoE models trained for 17.1–17.6 minutes vs. 11.6 for the dense model, despite fewer FLOPs per token. At this scale, gathering tokens into four small expert matrix multiplies is less GPU-efficient than one dense 2048-wide multiply.

1.4 Expert usage per language and distribution of labour

Figure 2 shows which experts each language activates; Table 2 summarises the balance. Four patterns stand out.

1. Routing does not depend on the source language. In every model and layer, the vi and ja columns are nearly identical. We measured the total-variation distance between the vi and ja routing distributions: at most 0.20 on source tokens (M4, layer 3) and at most 0.03 on English-prefix tokens. Even in layer 1, where Latin-script Vietnamese and Japanese kana/kanji have completely different embeddings, both go to the same expert (M2: E1 takes 97% of vi and 94% of ja source tokens). We had expected

per-language experts; the models instead learned experts that are shared across languages.

2. Experts specialise by token role. The split is between *source* tokens and *English* tokens. In M2 layer 1, E1 takes 97% of source tokens but only 3% of English tokens. In M5 layer 1, E3 is in the top-2 of 98–99% of source tokens but only 1% of English tokens. The router learns from the first layer onwards which side of `[SEP]` a token is on.

3. Labour is most uneven in early layers on the source side. Top-1 routing (M2) nearly collapses there (layer-1 entropy 0.11–0.18; layer 2 sends 78–85% of source tokens to E2). Deeper layers are much more even (entropy ≈0.8–0.9). With top-2 (M3), layer-2 source tokens use a nearly fixed pair (E2 in 99.9% and E1 in 92–94% of cases), which behaves like a dense layer. English-prefix tokens are spread more evenly in every model, and M5, whose experts are twice as wide, is the most balanced overall (mean source $H = 0.94$). In M4 the shared expert does the common work, and the routed experts are close to uniform on English tokens (layer-5 $H \approx 1.00$) but still skewed on source tokens.

4. No expert dies. Every expert receives tokens in every layer, language and region (0 unused cells out of 96), and final load-balancing losses were 1.01–1.08 (1.0 = uniform). The balancing loss works on all tokens of a batch together, so it prevents collapse in aggregate but does not stop experts from specialising by region, which is exactly what we observe.

2 Part 2: Optimizers written from scratch

2.1 Setup

Task and data. A causal language model is trained on `browndw/human-ai-parallel-corpus` (pinned revision, 66,320 documents that are human- and AI-written versions of the same sources). So that parallel versions of one document never sit in different splits, the corpus is split 90/5/5 by *document family* (the `doc_id` prefix): 59,688 / 3,312 / 3,320 documents. A new 16,000-token byte-level BPE tokenizer is trained on the training split only (0 unknown tokens in every split). Documents are wrapped in `<bos>`/`<eos>`, concatenated and packed into 256-token blocks. One pass over the 153,169 training blocks (39,211,264 prediction tokens) is the "1× dataset" budget.

Model and protocol. The architecture is Part 1's dense model (27,079,680 parameters; $d = 512$, 6 layers, 8 heads, FFN 2048, RoPE, tied embeddings, dropout 0.1). For a fair comparison, every optimizer starts from *the same saved initial weights* (hash-checked), sees the blocks in *the same order* (seed 42), and uses batches of $32 \times 256 = 8{,}192$ tokens, linear warm-up over 5% of tokens then cosine decay to 0.1× the peak LR, global gradient clipping at 1.0, decoupled weight decay and FP16 autocast with a gradient scaler. Only the optimizer and its LR change.

Evaluation at every 0.1× of the dataset. *Validation loss*: mean token cross-entropy over all 8,523 validation blocks. *Test BLEU*: for each of the 8,510 test blocks the model sees the first 128 tokens, greedily generates 64 tokens, and sacreBLEU compares them with the true next 64 tokens. As open-ended continuation with one reference, absolute BLEU is low (below 4); it serves to rank optimizers, not to measure translation quality.

2.2 Optimizers

All five subclass only `torch.optim.Optimizer`; no built-in update rule is called. g is the clipped gradient, η the scheduled LR and λ the weight decay; every rule first applies decoupled decay, $w \leftarrow (1 - \eta\lambda)w$.

- **AdamW** (baseline; $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, $\lambda = 0.1$): $m \leftarrow \beta_1 m + (1-\beta_1)g$, $v \leftarrow \beta_2 v + (1-\beta_2)g^2$, $w \leftarrow w - \eta\,\hat{m}/(\sqrt{\hat{v}} + \epsilon)$ with bias-corrected $\hat{m}, \hat{v}$.
- **NAdamW** (variance-reduced AdamW variant; same hyper-parameters): the same moments, but the update uses the Nesterov look-ahead $\tilde{m} = \beta_1 m + (1-\beta_1)g$ (bias-corrected), so the momentum is applied one step early.
- **Lion** (memory-efficient; $\beta = (0.9, 0.99)$, $\lambda = 0.3$): $w \leftarrow w - \eta\,\mathrm{sign}(\beta_1 m + (1-\beta_1)g)$, then $m \leftarrow \beta_2 m + (1-\beta_2)g$. It keeps one state tensor instead of two (108 MB instead of 217 MB here).
- **Muon** (matrix-based; momentum 0.95, $\lambda = 0.1$): for the 24 weight matrices inside the blocks (18.87M parameters), $m \leftarrow \beta m + g$, $u = \beta m + g$ (Nesterov), $w \leftarrow w - \eta\sqrt{\max(1, r/c)}\ \mathrm{NS}_5(u)$. NS_5 is five Newton–Schulz iterations $X \leftarrow 3.4445X - 4.7750(XX^\top)X + 2.0315(XX^\top)^2X$ on the normalised u, which approximately orthogonalise the update (singular values → 1). The tied embedding and the LayerNorm vectors (8.21M parameters) use the hand-written AdamW rule with LR $6 \cdot 10^{-4}$.

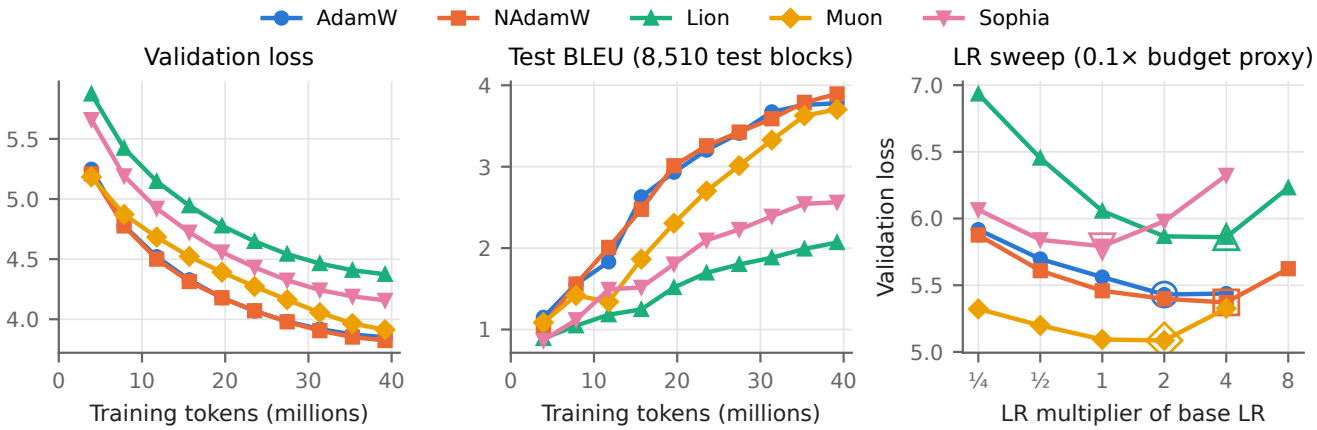


Figure 3: Part 2. Validation loss (left) and test BLEU (middle) at every 0.1× of the dataset; learning-rate sweep on the proxy budget (right, large hollow markers = selected LR).

Table 3: Part 2 at half and full budget. Wall clock is optimisation time only (excluding evaluation). Peak memory includes evaluation. AMP skips are steps skipped by the FP16 gradient scaler.

	Selected LR	Val. loss		Val. PPL	Test BLEU		Train time	Peak mem.	AMP
Optimizer	(multiplier)	0.5×	1.0×	1.0×	0.5×	1.0×	(s)	(GiB)	skips
AdamW	0.0012 (2×)	4.179	3.849	47.0	2.93	3.78	370	4.69	1
NAdamW	0.0024 (4×)	4.179	3.824	45.8	3.01	3.89	390	4.69	1
Lion	0.0004 (4×)	4.777	4.374	79.3	1.52	2.07	359	4.59	0
Muon	0.04 (2×)	4.392	3.913	50.1	2.30	3.70	631	4.62	0
Sophia (bonus)	0.0003 (1×)	4.559	4.157	63.9	1.80	2.56	414	4.92	0

- **SophiaG** (bonus, Hessian approximation; $\beta = (0.965, 0.99)$, $\rho = 0.04$): $m \leftarrow \beta_1 m + (1-\beta_1)g$. Every 10 steps the diagonal Hessian estimate h is refreshed with the Gauss–Newton–Bartlett estimator: sample labels from the model's own softmax, back-propagate, and set $h \leftarrow \beta_2 h + (1-\beta_2)\hat{g}^2$. The update is $w \leftarrow w - \eta \operatorname{sign}(m) \min(|m|/(\rho B h + \epsilon), 1)$, with $B = 8192$.

Unit tests check AdamW, NAdamW and Lion against scalar re-implementations of these equations, Muon's step against Nesterov momentum + Newton–Schulz + shape scaling, Newton–Schulz against the exact polar factor from an SVD, and Sophia's clipped ratio against a hand computation.

2.3 Learning-rate sweep (bonus)

Each optimizer's peak LR was tuned with proxy runs on 0.1× the data (3,920,896 tokens), each with its own complete warm-up and cosine schedule, from the shared initial weights. The grid was multipliers $\{\frac{1}{4}, \frac{1}{2}, 1, 2, 4\}$ of a base LR (AdamW/NAdamW $6 \cdot 10^{-4}$, Lion 10^{-4}, Muon 0.02, Sophia $3 \cdot 10^{-4}$), extended by 2× whenever the best value lay on an edge. Selection used validation loss only; the test set was never used. NAdamW and Lion each needed one extension, to 8×. Every curve in Figure 3(right) is U-shaped with its minimum inside the grid. **Muon** is the least sensitive: 5.09–5.33 over a 16× LR range, and below every other optimizer's best value at every LR it tried. **Lion** is the most sensitive: 6.93 at $\frac{1}{4}$× down to 5.86 at 4×. **Sophia** degrades quickly above its base LR (6.32 at 4×).

2.4 Results

Figure 3 and Table 3 give the results. At the full budget the order is **NAdamW** < **AdamW** < **Muon** < **Sophia** < **Lion** in validation loss, and exactly the reverse in test BLEU. BLEU rises with training for every optimizer (Muon has a single dip, at 0.3×) and tracks validation loss closely, which suggests continuation BLEU is a usable, if coarse, proxy here.

AdamW vs. NAdamW. The two curves almost overlap. NAdamW ends slightly ahead (3.824 vs. 3.849; BLEU 3.89 vs. 3.78) with twice the LR ($2.4 \cdot 10^{-3}$ vs. $1.2 \cdot 10^{-3}$). This fits the Nesterov look-ahead tolerating a larger step. With a single seed, a 0.025-nat gap is too small to call a real difference.

Muon is the fastest optimizer at the start: it has the lowest loss at 0.1× (5.184 vs. 5.209 NAdamW and 5.247 AdamW) and the best proxy losses in the sweep. From 0.2× onwards, however, the Adam-family optimizers overtake it and stay ahead (3.913 vs. 3.849 at the end). This matches the observation of Wen et al. (2025, arXiv:2509.02046) that early-training comparisons can mislead because rankings

change as the LR decays. Two likely reasons for Muon's weaker finish are specific to our setup. First, 30% of the parameters, including the large tied embedding/LM head, use the fallback AdamW with an LR ($6 \cdot 10^{-4}$) that was not swept and is half of AdamW's tuned LR. Second, our budget is only about 1.4 tokens per parameter. Muon also costs the most time: 631 s vs. 370 s for AdamW (+70%), because of the five FP32 Newton–Schulz iterations per matrix per step.

Lion is the weakest, partly because of the sweep. Extra full-budget runs outside the sweep protocol (same initial weights and data order) reached 4.297 at LR 10^{-4} and 4.256 at $2 \cdot 10^{-4}$, both better than 4.374 at the selected $4 \cdot 10^{-4}$ (BLEU 2.33 and 2.43 vs. 2.07). The proxy compresses the whole schedule into 10% of the steps, which rewards aggressive LRs. Lion's sign update gives every coordinate the same step size, so it is hit hardest when the LR is too high. Even at its best LR, Lion stays behind Sophia. Its memory saving is real but small in practice: peak memory is 4.59 vs. 4.69 GiB, because activations dominate at this model size.

Sophia lands between Muon and Lion. Its logged statistics explain why: 93.0–94.2% of coordinates hit the clip $\min(\cdot, 1)$ throughout training, so the update is mostly $\eta\,\mathrm{sign}(m)$, close to signed momentum like Lion, and curvature only rescales about 6% of coordinates. A larger ρ would let the Hessian matter more but was not tuned. The Hessian refresh adds an extra backward pass every 10 steps: training takes 12% longer than AdamW (414 s) and Sophia has the highest peak memory (4.92 GiB).

3 Part 3: Decoding strategies

3.1 Setup and implementation

Model and data. `EleutherAI/pythia-160m` (pinned, FP32) on the whole ROCStories test split (`hamishivi/ROCStories`, pinned): 19,611 stories, after excluding 22 rows whose reference continuation is empty. The prompt is the story's first sentence; the reference is the remaining four sentences (mean 42.2 Pythia tokens). Each strategy generates at most 64 tokens and stops at EOS. Newline tokens (188 ids) are banned for every strategy because a story is a single paragraph.

Decoding from `model.forward()`. `generate()` is never used (the experiment sets it to `None`). Prompts are left-padded with explicit position ids, and every step feeds one new token per row through the model with a KV cache.

- **Greedy**: argmax of the masked logits.
- **Beam search** (width k): each prompt's cache is copied k times, and every beam except the first starts at $-\infty$. At each step, the $2k$ best of the $k \cdot V$ extensions are ranked by cumulative log-probability. An EOS extension ranked in the top k becomes a finished hypothesis; the others fill the k live beams, and the cache is reordered to match. A prompt finishes once it has k finished hypotheses. The winner maximises score/length$^{\alpha}$ with $\alpha = 1$.
- **Top-k**: logits below the k-th largest are set to $-\infty$. **Top-p**: the smallest set of tokens, in descending order, whose mass reaches p, including the token that crosses p, so the most likely token is always kept. **Pure sampling**: the full softmax at temperature 1.

Sampling runs use seeds 0, 1, 2. Unit tests check that cached left-padded decoding equals uncached decoding, that beam-1, top-k=1 and top-$p\to$0 equal greedy, that exhaustive beam search finds the most probable sequence, and that beam results do not depend on batching.

Metrics (per story, then averaged). *Perplexity*: $\exp(\sum \mathrm{NLL}/\#\mathrm{tokens})$ of a strategy's own output (including EOS) under the unmodified model. The human reference continuations score **24.71**. *Accuracy*: position-by-position match between generated and reference Pythia tokens, divided by the reference length. *Precision/recall/F1*: overlap between the multisets of lower-cased, punctuation-free words of the generation and of the reference. We also record corpus-level distinct-n, the fraction of trigrams that repeat earlier in the same text, and the fraction of outputs ending in EOS.

3.2 Results

Table 4 and Figure 4 summarise the metrics, and Table 5 shows one story.

Deterministic decoding degenerates. Greedy and beam outputs have very low perplexity (2.19 greedy, 1.90 beam-4) because they loop. 71–74% of their trigrams repeat earlier ones, and distinct-2 is only 0.028–0.029. In Table 5 greedy repeats one sentence and beam-4 a clause; story 0 behaves the same way ("He was a great mentor to me." ×6). A wider beam finds higher-likelihood text, but that text

Table 4: Part 3 on 19,611 ROCStories test stories; sampled rows are means over 3 seeds (± std; std ≤ 0.0015 elsewhere). Gen. PPL: model perplexity of its own output (human reference 24.71). Rep-3: repeated-trigram fraction. Len: mean tokens. Stop: fraction ending in EOS. Time: full-run decode seconds (batch 128 greedy/sampling, 32 beam).

Strategy	Accuracy	Prec.	Recall	F1	Gen. PPL	Distinct-2	Rep-3	Len	Stop	Time (s)
greedy	0.0289	0.111	0.173	0.1333	2.19	0.028	0.714	63.9	0.003	132
beam_1	0.0289	0.111	0.173	0.1334	2.19	0.028	0.714	63.9	0.003	513
beam_2	0.0288	0.108	0.168	0.1299	2.05	0.028	0.723	63.8	0.013	609
beam_4	0.0267	0.100	0.152	0.1187	1.90	0.029	0.742	63.4	0.043	791
top_k_5	0.0208±0.0000	0.161	0.248	0.1920±0.0002	6.43±0.01	0.102	0.096	62.8	0.047	146
top_k_20	0.0166±0.0001	0.161	0.245	0.1911±0.0004	11.18±0.00	0.160	0.020	62.0	0.065	146
top_k_50	0.0152±0.0002	0.157	0.236	0.1850±0.0003	15.32±0.01	0.203	0.010	61.9	0.066	146
top_p_0.5	0.0198±0.0002	0.157	0.238	0.1860±0.0003	6.77±0.00	0.126	0.104	62.7	0.043	176
top_p_0.9	0.0137±0.0002	0.147	0.214	0.1713±0.0005	27.46±0.09	0.325	0.005	62.1	0.059	175
top_p_0.95	0.0128±0.0002	0.141	0.203	0.1639±0.0007	38.92±0.18	0.381	0.003	62.1	0.060	175
pure_sampling	0.0119±0.0001	0.134	0.189	0.1539±0.0001	68.17±0.35	0.462	0.002	62.1	0.062	138

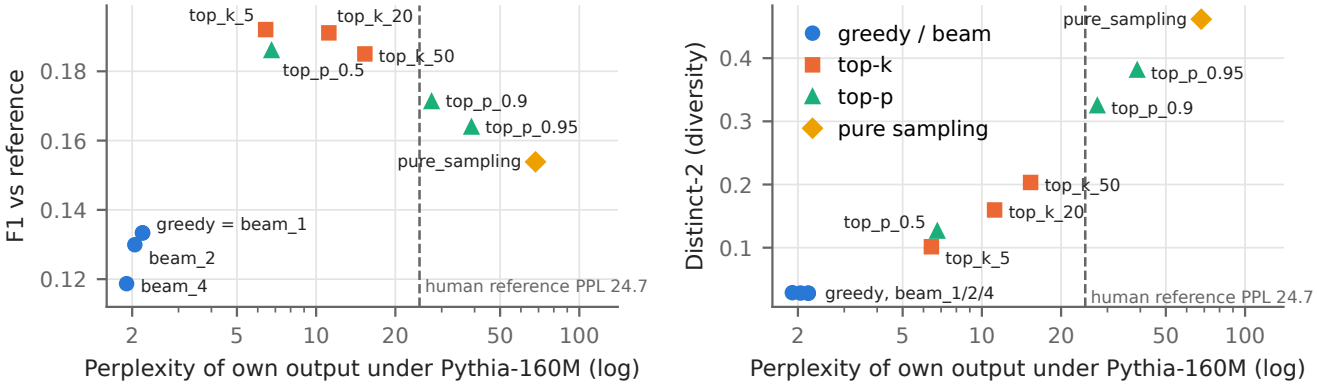


Figure 4: Each strategy's F1 against the reference (left) and diversity (distinct-2, right), plotted against the perplexity of its own outputs (log scale). The dashed line marks the human reference perplexity (24.71).

is *more* repetitive (Rep-3 0.714→0.742) and further from the reference (F1 0.133→0.119). Likelihood and quality diverge here: the most probable continuation under the model is a loop, not a story (the degeneration described by Holtzman et al., 2020). The model also almost never emits EOS (stopped fraction 0.3–4.3%), so most outputs run the full 64 tokens, about 1.5 times the average reference length.

Truncated sampling gives the best content overlap. Every sampling method has higher precision, recall and F1 than every deterministic one. Top-k=5 is best (F1 0.192), followed by top-k=20 (0.191). Widening the candidate set (k: 5→50, p: 0.5→0.95→1) raises diversity monotonically (distinct-2 0.10→0.46) and raises output perplexity from 6.4 to 68.2, while F1 falls steadily. Top-p=0.9 (27.5) lands closest to the human perplexity of 24.7, and top-k=50 (15.3) is the next closest. In Table 5, top-k=50 keeps the skiing topic ("ski jumping"); top-p=0.9 and pure sampling drift into unrelated first-person narratives with invented details ("Canada in 2007", "Victoria Reservoir"). Top-p 0.9 and pure sampling begin identically because both use seed 0, and hence the same random numbers; seed std is tiny (≤0.0007 in F1).

Accuracy is near the floor for every strategy (1.2–2.9%). Position-wise matching against one human continuation punishes any one-token shift; deterministic decoding ranks first only because high-probability tokens line up with the reference most often. Bag-of-words F1 is the more meaningful overlap metric here, and even it stays below 0.2 because many continuations are valid.

3.3 Beam-search time and complexity

Complexity. Let k be the beam width, T the number of generated tokens, V the vocabulary, C the cost of one token through the network, and L, d the number of layers and the model width. Each of the T steps runs one forward pass on k rows, costing $O(k\,(C + L\,d\,t))$ at step t with a KV cache. It then scores $k \cdot V$ extensions with a log-softmax and a top-$2k$ selection ($O(kV)$) and reorders the cache ($O(k\,L\,d\,t)$). In total, beam search costs

$$O(T\,k\,C \;+\; k\,L\,d\,T^2 \;+\; T\,k\,V) \text{ time}, \qquad O(k\,L\,d\,(n_{\text{prompt}} + T) + kV) \text{ memory},$$

Table 5: Story 1 of the test split (verbatim, cut at "…"). Sampled outputs use seed 0.

Prompt	Dave went to Mammoth Mountain to go skiing.
Reference	He put on his boots, skis and goggles. With great excitement, he climbed into the chair lift. There was a foot of fresh powder from a recent storm. Dave jumped off the chair lift and took off down the mountain.
Greedy	He was a good skier, but he was a bit of a skier. He was a good skier, but he was a bit of a skier. He was a good skier, but he was a bit of a skier. He …
Beam-4	The first thing he did was go to the top of the mountain, and then he went back to the top of the mountain. Then he went back to the top of the mountain, and then he went back to …
Top-k 50	His mother said he was great at taking care of his girlfriend, who told him in the last scene that he was born in the park, so he was good at ski jumping, in this scene of the little house …
Top-p 0.9	His grandma did the thing before, and we had things done there too, in the way I went to Canada in 2007. We went for short walks, and put on my gloves. The first day we drove up on March …
Pure	His grandma did the thing before, and we had things done there too, in the safe haven of a place known as the Victoria Reservoir. Last semester (other than in this old-school TV house salvage place) …

Table 6: Beam timing on a fixed 256-story subset (CUDA-synchronised, after a warm-up). "Rows" is the number of sequences per forward call.

	1 prompt per batch			32 prompts per batch			
Width k	ms/forward	ms/story	peak MiB	rows	ms/forward	ms/story	peak MiB
1	10.02	641	635	32	12.94	25.9	838
2	10.23	652	641	64	15.32	30.6	1049
4	10.38	657	654	128	19.82	39.6	1470
8	11.06	705	680	256	33.62	67.2	2313
16	10.90	696	733	512	64.67	129.3	3999

which is *linear in* k: k times the work of greedy decoding ($k = 1$), but with the same number T of sequential steps.

Measurements (Table 6; plot in `report/figures/part3/beam_width_timing.png`). The linear term only shows once the GPU is busy. With one prompt per batch, a forward call costs 10.0–11.1 ms at every width from 1 to 16: the 160M-parameter model leaves the GPU under-used, so step latency, not FLOPs, sets the time, and 16 beams are almost free. With 32 prompts per batch, the cost per call grows by 1.18×, 1.29×, 1.70× and 1.92× for each doubling of k, approaching the predicted 2× once there are 256–512 rows per call. Memory is exactly linear: each extra beam row costs 6.59 MiB at every step of the grid. That is of the size expected for one row's KV cache (2×12 layers $\times 768 \times 4$ bytes = 72 KiB per cached token, over the padded prompt plus up to 64 generated tokens) plus its V-wide logit rows. On the full test split, beam widths 1/2/4 took 513/609/791 s (1.19×, 1.30× per doubling), in line with the subset. Batching matters far more than width: at $k = 1$, 32 prompts per batch cut the time per story from 641 ms to 25.9 ms.

Greedy vs. beam-1. The two are equal in theory (the unit tests confirm it), yet full-run greedy took 132 s and beam-1 513 s. This comes entirely from the batch size (128 vs. 32): the cost per forward call is the same (13.4 vs. 13.1 ms), but beam-1 needs four times as many calls. Their outputs agree on 19,522 of 19,611 stories. At all 89 first divergences the two competing tokens are the model's top two, with a median log-probability gap of $3.7 \cdot 10^{-4}$: floating-point near-ties, not a decoder bug.

4 Conclusions

MoE: at equal total parameters, sparse FFNs give up 2–3% perplexity for 23–35% fewer active parameters; at equal active parameters, extra experts (M5) give the best model. Routers specialise by token role and depth, not by language, and the balancing loss keeps every expert in use. **Optimizers:** tuned AdamW/NAdamW are strongest at this budget; Muon leads early but costs 70% more time; Sophia acts mostly like signed momentum (∼94% of coordinates clipped); short-budget LR sweeps can mislead (Lion). **Decoding:** greedy and beam search fall into repetition loops, which widen with the beam; truncated sampling (top-k 5–50, top-p 0.5–0.9) best matches human continuations; beam search is linear in k, but on a GPU its wall-clock cost depends mostly on batching.

Code: `part 1/`, `src/part2`, `src/part3`; results and figures in `report/`. *LLM use:* drafted with Claude from the project's code and results, as the course announcement permits.